EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

Declarative programming always requires the programmer to specify every loop and update.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

DO NOW — RECALL

Lesson 123 · Programming Paradigms: How We Express

Name: _____ Class: _____ Date: _____

1. What development approach is most explicitly risk-driven?

2. What is the effect of enqueue(X), enqueue(Y), dequeue() on an initially empty FIFO queue?

3. What is the time complexity of sequential search in the worst case?

4. After pushing A then B onto a stack, which is popped first?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Spiral.

2. Model answer

X is removed; Y remains at the front.

3. Model answer

O(n).

4. Model answer

B, because a stack is LIFO.

By the end, you can…

- Express a sequence of state-changing instructions and reusable procedures.
- Describe a desired result or logical relationship rather than an explicit control-flow recipe.
- Model collaborating objects with state/behaviour; pick style for the problem.
- Explain inheritance, overriding and polymorphic method calls using C#.

Vocabulary — know these terms

paradigm — Style or model for expressing computations.

imperative — Programming by explicit state-changing commands.

declarative — Programming by describing desired result/relations.

object-oriented — Programming using interacting objects with state and behaviour.

inheritance — A derived class reuses and specialises a base class.

override — A derived class replaces an inherited virtual method implementation.

polymorphism — One base-class method call can run different implementations according to the actual object type.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

A base class Alert declares virtual Show(). Two derived classes EmailAlert and PopupAlert override Show(). An Alert[] contains one of each. Explain inheritance, overriding, polymorphism and one reason this design may be preferable to procedural type-checking. [8]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Define a programming paradigm and contrast imperative with declarative languages. [6]

Question 2

Explain how a SQL query differs conceptually from a loop-based filter. [6]

NEW LEARNING

Read, annotate and explain

Imperative/procedural

Express a sequence of state-changing instructions and reusable procedures.

Declarative

Describe a desired result or logical relationship rather than an explicit control-flow recipe.

Object-oriented

Objects group state and behaviour. Inheritance lets a derived class reuse a base class; an override replaces a virtual method. Calling that method through a base-class reference runs the actual object’s version: polymorphism .

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Compare a SQL query selecting all orders over £100 with a loop that scans each record.

Model answer

SQL describes the required set of matching records (declarative); the loop specifies iteration, testing and append operations (imperative/procedural). Both can produce equivalent output through different expression styles.

Check: link each claim to the scenario and use precise OCR terminology.

Find the parent and children

Alert is the base class; EmailAlert : Alert and PopupAlert : Alert are derived classes. Each is a kind of Alert and can be stored in an Alert[] .

Same signature, specialised behaviour

Show() is virtual in Alert . Both derived classes use override to supply their own implementation.

Trace the output

a.Show() prints Email then Popup . Although a is declared as Alert , the runtime selects the implementation belonging to the actual object. This is polymorphism, not two separate if tests.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve